# Yolov5 Model Training

## 1. Image Collecting & Labeling

Given that training the Yolov5 model necessitates a substantial volume of data, our initial step involves collecting and labeling the requisite data in preparation for subsequent model training.

To begin, you can assemble the images that require collection. As an illustration, let's consider the collection of traffic signs.

### 1.1 Image Collecting

1) Start the robot, and access the robot system desktop using VNC.

2) Click-on  to open the command-line terminal.

3) Execute the command to terminate the app auto-start service.

```
~/.stop_ros.sh
```

```
~/.stop_ros.sh
```

4) Run the command to initiate the depth camera service.

```
ros2 launch peripherals depth_camera.launch.py
```

```
ros2 launch peripherals depth_camera.launch.py
```

5) Navigate to the path of image acquisition program, execute the command to open the image acquisition tool.

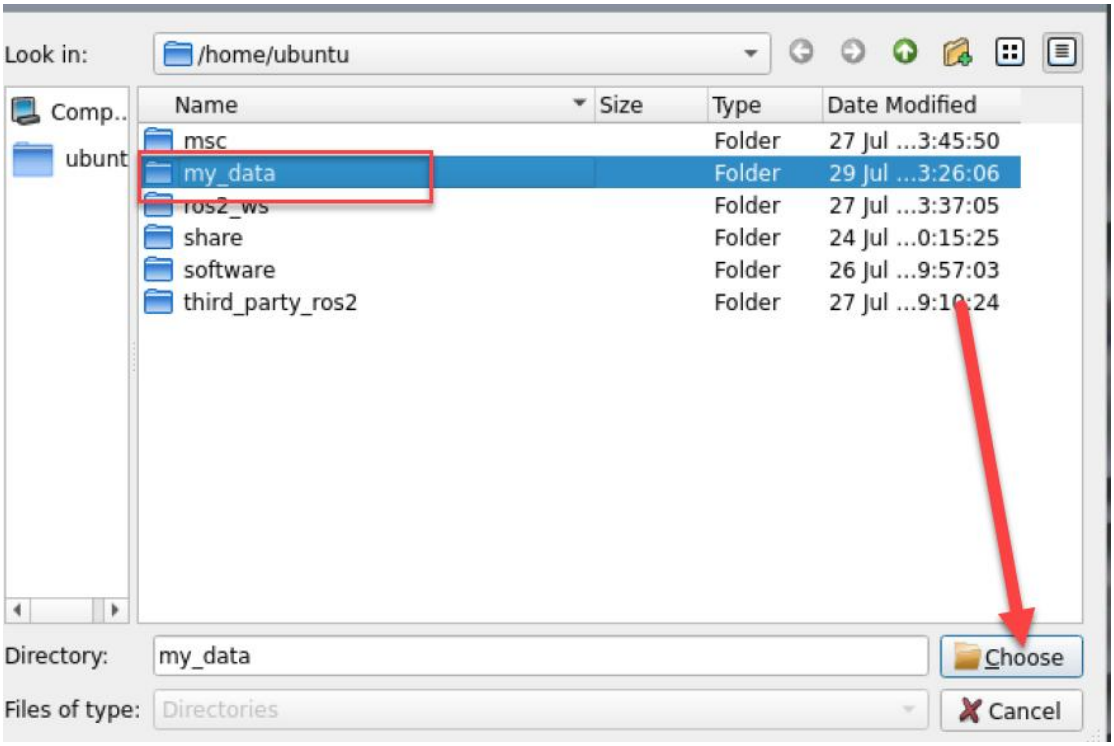
```
cd ~/software/collect_picture && python3 main.py
```

```
cd ~/software/collect_picture && python3 main.py
```


The "save number" displayed in the upper left corner represents the image ID, indicating the order in which images are saved. The term "existing" denotes the total number of images already saved.

6) Click “Select”. Change the storage path to the created folder directory.


7) After selecting, click “Choose”.




8) Position the target recognition content within the camera's field of view, then either click the "Save" button or press the space bar to capture the current image from the camera.

Click-on “Save(space)” or press space bar, then JPEGImages folder will be generated to save pictures.

Note: To enhance model reliability, capture target recognition content from various distances, rotation angles, and tilt angles.

9) Once you have finished collecting the images, click-on “Exit” button to close the software.


10) In all the open command line terminals, click the “Ctrl+C” to exit. At this time, the entire progress of image acquisition is completed.

11) Create a folder by inputting the command:

```
touch ~/software/collect_picture/my_data/JPEGImages/classes.txt
```

```
) touch ~/software/collect_picture/my_data/JPEGImages/classes.txt
```

## 1.2 Image Labeling

Labeling images is crucial for feature datasets as it enables the trained model to understand the categories of significant elements within the image. This understanding empowers the model to recognize these categories in new, previously unseen images.

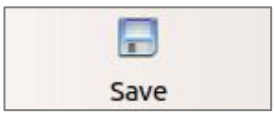
Note: the input command should be case sensitive, and keywords can be complemented using Tab key.

1) Open the command-line terminal, and execute the command to open the image labeling software.

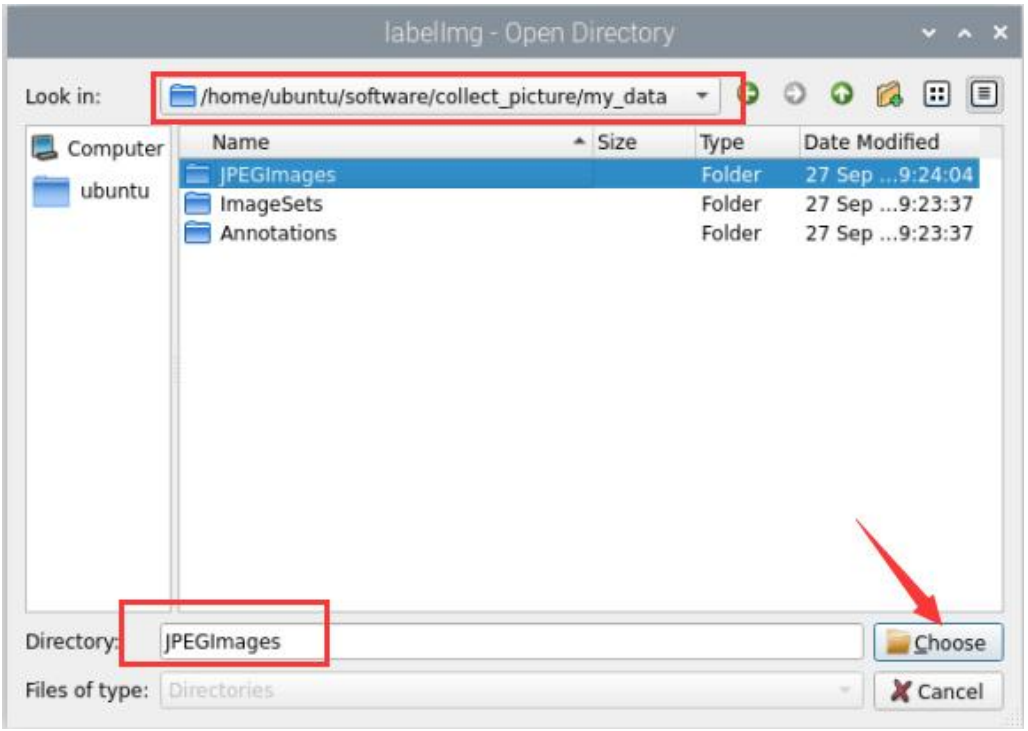
```
python3 ./software/labelImg/labelImg.py
```

```
> python3 ~/software/labelImg/labelImg.py
```

The table below outlines the function of each icon:

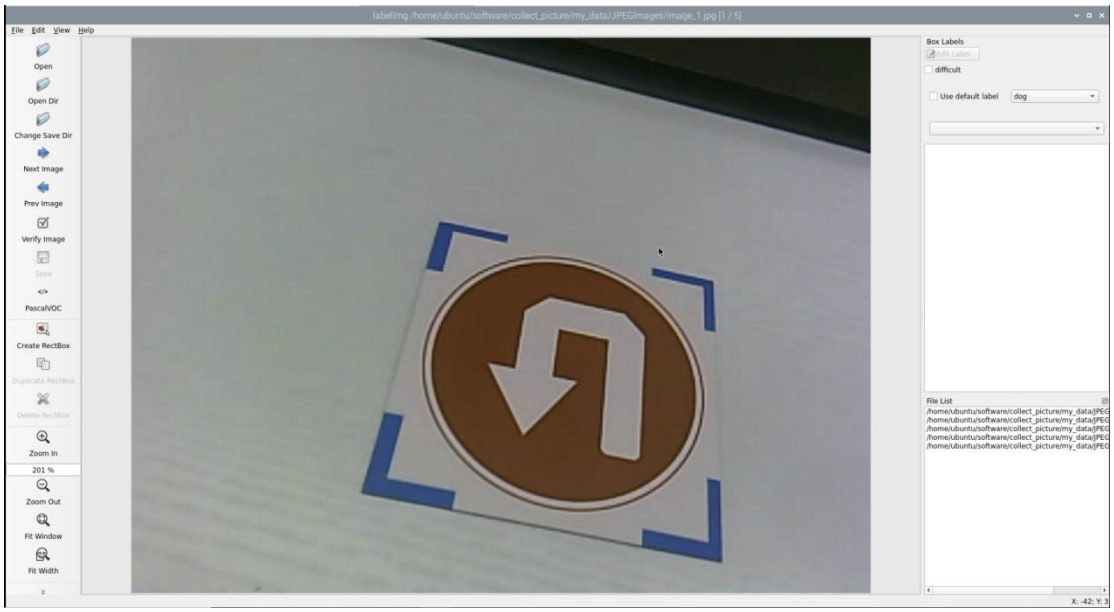
| Icon | Shortcut Key | Function |
|---|---|---|
|  | Ctrl+U | Select the directory where the picture is saved. |
|  | Ctrl+R | Select the directory where the calibration data is saved. |
|  | W | Create annotation box |
|  | Ctrl+S | Save annotation |
|  | A | Switch to the previous image |
|  | D | Switch to the next image |

2) Click  to open the directory where the captured images are saved. In this section, it is as below:



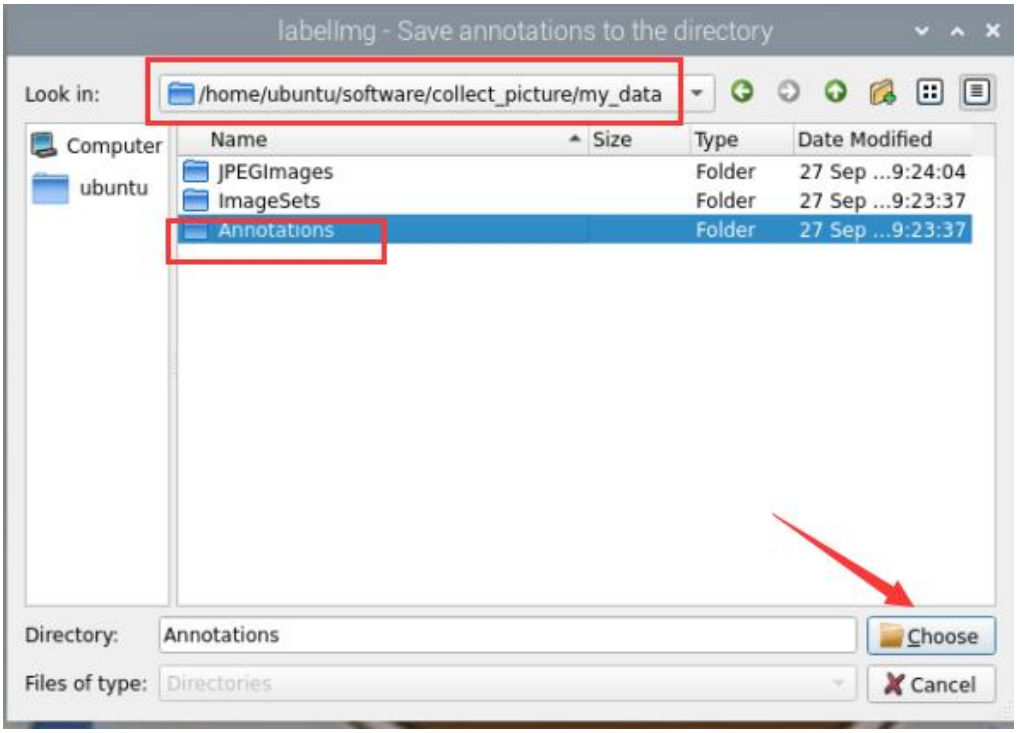
3) After clicking 

the “choose” button , the directory will be opened as shown in the figure.



4) Click the 

on the interface. Select the corresponding storage folder for the calibration data directory. The path is the "Annotations" folder in the same path as the image capture.

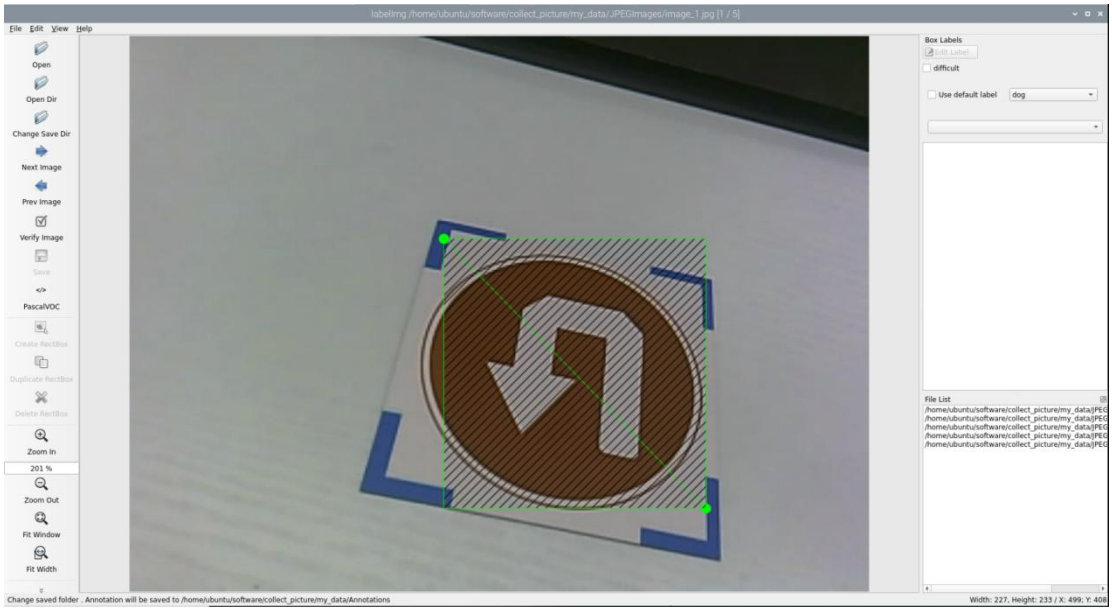


5) Click 

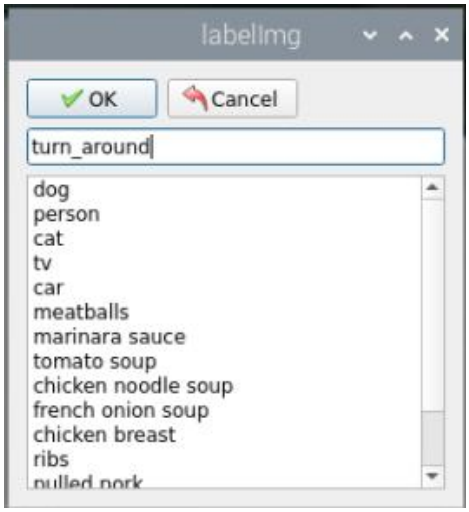
to return to the image annotation interface.

6) Press the "**W**" key on the keyboard to create the label box.

7) Position the mouse cursor appropriately, then press and hold the left mouse button while dragging it to encompass the entire target recognition content within the label box. Release the left mouse button to finalize the selection of the target recognition content.

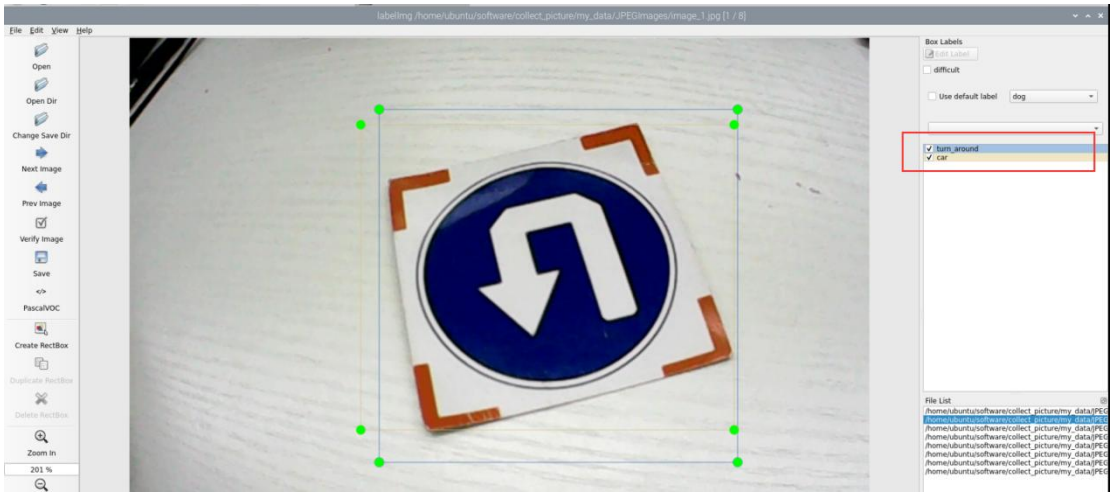



8) In the pop-up window, assign a category name to the target recognition content; for instance, "turn_around". Once you've named it, either click the "OK" button or press the "Enter" key to save this category.



9) Press the shortcut “Ctrl+S” to save the labeling data for the current picture.

10) As shown in the image, the right side of the image annotation is the label recognition box management area. In the red highlighted section, you’ll notice multiple recognition boxes placed on the same image. By ticking the checkbox next to a specific label, the corresponding recognition box will appear and become active effect; unticking it will hide the box and deactivate it.




11) Press the "d" key to enter the annotation of the next image. Repeat steps 7 to 9 to complete the annotation of all images. Click  in the upper right corner of the software to exit.

12) Open a new command line. Run the command to view the annotation files.

```
cd software/collect_picture/my_data/Annotations && ls
```

# 2. Data Format Conversion

## 2.1 Getting Ready

Before proceeding with this section, ensure that image collection and annotation have been completed. For detailed operational steps, please refer to the "1.Image Collecting and Labeling".

Prior to feeding the data into the YOLOv5 model for training, it is essential to assign categories to the images and convert the annotated data into the required format.

## 2.2 Format Conversion

Please operate the below steps after collecting and labeling the pictures.

---

Note: the input command should be case sensitive, and keywords can be complemented using Tab key.

---

1) Execute the following command to write the category for the captured images from this training round into a specific text file.

```
gedit software/collect_picture/my_data/classes.names
```


2) Press “i” to enter the editing mode, then write the labeled category, “**turn_around**”, into the text file (For multiple different categories, press Enter to start a new line for each one).


3) After completing the editing, press the "ESC" key, type ":wq", and press Enter to save and exit.

---

Note: The class name added here must match the one used in 'labelImg'.

---

4) Return to the command line terminal. Execute the command to convert the

data format.

```
python3 software/collect_picture/xml2yolo.py --data
software/collect_picture/my_data --yaml
software/collect_picture/my_data/data.yaml
```

```
) python3 software/collect_picture/xml2yolo.py --data software/collect_picture/m
y_data --yaml software/collect_picture/my_data/data.yaml
data: software/collect_picture/my_data
turn_around:5
{'train': 'software/collect_picture/my_data/ImageSets/train.txt', 'val': 'softwa
re/collect_picture/my_data/ImageSets/val.txt', 'nc': 1, 'names': ['turn_around']
}
finish!
```

---

Note: the path of “~/software/xml2yolo.py” and “my_data” must be determined according to the actual location of the file.

---

In the command above, three parameters are involved:

**a. xml2yolo.py:** This script converts the calibrated label format into a format compatible with YOLOv5 model conversion. Ensure the path corresponds correctly.

**b. my_data:** This parameter specifies the folder you have labeled. Ensure the path corresponds correctly.

**c. data.yaml：** This file represents the format conversion of the entire folder after the model segmentation. As indicated in the command, the saved directory is within the my_data folder.

**d.** The image below illustrates the generated data.yaml file.

```
1 names:
2 - turn_around
3 nc: 1
4 train: software/collect_picture/my_data/ImageSets/train.txt
5 val: software/collect_picture/my_data/ImageSets/val.txt
```

The points after the names represent the label type. 'nc' denotes the number of label types, 'train' signifies the training set (used for data training in deep learning), and the subsequent parameters denote the paths. 'val' represents the validation set, which is used to verify results during the data training process. Ensure that paths are filled in or replaced according to the actual

locations.

In subsequent training processes, if you need to enhance training speed by moving the dataset from the vehicle to a local PC or cloud server, ensure to replace the paths corresponding to the current 'train' and 'val' datasets.

Finally, an XML file will be generated under the "/my_data" folder to locate the segmented dataset. You can also change the saved path by modifying the last parameter in the 4) command "/my_data/data.yaml". Remember the path of this file as it will be used for model training later.

5) Return to the terminal,then enter the command to edit the yaml file and press "**Enter**."

```
gedit software/collect_picture/my_data/data.yaml
```


6) Change the relative paths of the train and val files in the configuration file to absolute paths in the yaml file.

The relative paths for the training set (train) and validation set (val) in this file are relative to the directory where the command in step 4 is executed.

Later, the commands from different directories will be executed to call this configuration file, which in turn will access the train and val files based on the paths defined within it.

If the paths remain relative, the system will interpret them based on the current working directory where the command is executed. This can lead to incorrect file locations and ultimately cause errors when the files cannot be found.

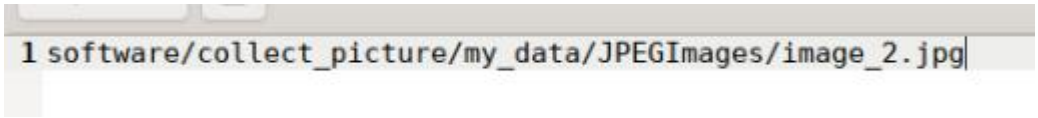
```
names:
- turn_around
nc: 1
train: /home/ubuntu/software/collect_picture/my_data/ImageSets/train.txt
val: /home/ubuntu/software/collect_picture/my_data/ImageSets/val.txt
```

**Note:The command for checking the absolute path of a file (Must be run in the directory where the file is located): pwd**

7) Return to the terminal, enter the command to edit the validation set file (val.txt), and press "**Enter**."

```
gedit software/collect_picture/my_data/ImageSets/val.txt
```


Before modification:




After modification:

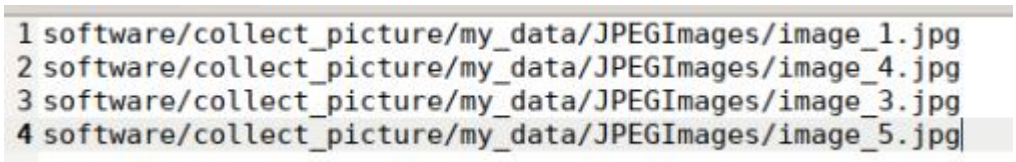

During model training, the previously executed program will divide the JPG image data collected during the "Image Collection" stage into the training and validation sets according to a certain ratio. These sets will then be used for subsequent model training and evaluation.

The training and validation set files contain the relative paths to the corresponding image files. To align with Step 6), these relative paths need to be converted to absolute paths.

1) Similarly, enter the command to edit the **train.txt** file and press Enter.

```
gedit software/collect_picture/my_data/ImageSets/train.txt
```


Before modification:



After modification:

```
/home/ubuntu/software/collect_picture/my_data/JPEGImages/image_1.jpg
/home/ubuntu/software/collect_picture/my_data/JPEGImages/image_4.jpg
/home/ubuntu/software/collect_picture/my_data/JPEGImages/image_3.jpg
/home/ubuntu/software/collect_picture/my_data/JPEGImages/image_5.jpg
```

**Note**: **If an error similar to the one shown below occurs after executing Step 4) in Section 3.2, please go back to Steps 7) and 8) in Section 2.2 to verify whether they were executed correctly.**


# 3. Model Training

Note: the input command should be case sensitive, and the keywords can be complemented using Tab key.

## 3.1 Getting Ready

After converting the model format, the next step is to proceed with model training. However, before training, ensure that the dataset has been prepared and converted into the required format. For detailed instructions, please refer to "2. Data Format Conversion".

## 3.2 Model Training

1) Start the robot, and access the robot system desktop using VNC.

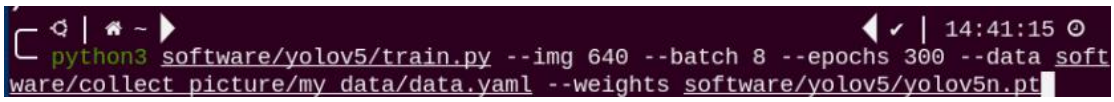
2) Click-on  to open the command-line terminal.

3) Execute the command to navigate to the designated directory and extract the Yolov5 installation package.

```
unzip software/yolov5.zip -d software/
```


4) Run the command to train model.

```
python3 software/yolov5/train.py --img 640 --batch 8 --epochs 300 --data software/collect_picture/my_data/data.yaml --weights software/yolov5/yolov5n.pt
```




In the command, "**--img**" specifies the image size; "**--batch**" determines the number of single input images per batch; "**--epochs**" indicates the number of training iterations; "**--data**" denotes the path to the dataset; "**--weights**" represents the location of the initial training file.

You can adjust these parameters according to your specific requirements. For instance, to enhance model reliability, one can increase the number of training iterations, though this will also extend the training duration.

If the output shown below appears, it indicates that your current network environment is unstable. Please try switching to a network with access to internet connection before proceeding.

```
❯ python3 software/yolov5/train.py --img 640 --batch 8 --epochs 300 --data software/collect_picture/my_data/data.yaml --weights yolov5n.pt
train: weights=yolov5n.pt, cfg=, data=software/collect_picture/my_data/data.yaml, hyp=software/yolov5/data/hyps/hyp.scratch-low.yaml, epochs=300, batch_size=8, imgsz=640, re
ct=False, resume=False, nosave=False, noval=False, noautoanchor=False, noplots=False, evolve=None, evolve_population=software/yolov5/data/hyps, resume_evolve=None, bucket=,
cache=None, image_weights=False, device=, multi_scale=False, single_cls=False, optimizer=SGD, sync_bn=False, workers=8, project=software/yolov5/runs/train, name=exp, exist_o
k=False, quad=False, cos_lr=False, label_smoothing=0.0, patience=100, freeze=[0], save_period=-1, seed=0, local_rank=-1, entity=None, upload_dataset=False, bbox_interval=-1,
 artifact_alias=latest, ndjson_console=False, ndjson_file=False
github: skipping check (not a git repository), for updates see https://github.com/ultralytics/yolov5
YOLOv5 🚀 2024-2-1 Python-3.10.12 torch-2.3.1 CPU

hyperparameters: lr0=0.01, lrf=0.01, momentum=0.937, weight_decay=0.0005, warmup_epochs=3.0, warmup_momentum=0.8, warmup_bias_lr=0.1, box=0.05, cls=0.5, cls_pw=1.0, obj=1.0,
 obj_pw=1.0, iou_t=0.2, anchor_t=4.0, fl_gamma=0.0, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, degrees=0.0, translate=0.1, scale=0.5, shear=0.0, perspective=0.0, flipud=0.0, fliplr=
0.5, mosaic=1.0, mixup=0.0, copy_paste=0.0
Comet: run 'pip install comet_ml' to automatically track and visualize YOLOv5 🚀 runs in Comet
TensorBoard: Start with 'tensorboard --logdir software/yolov5/runs/train', view at http://localhost:6006/
Downloading https://github.com/ultralytics/yolov5/releases/download/v7.0/yolov5n.pt to yolov5n.pt...
ERROR: <urlopen error [Errno -3] Temporary failure in name resolution>
Re-attempting https://github.com/ultralytics/yolov5/releases/download/v7.0/yolov5n.pt to yolov5n.pt...
curl: (6) Could not resolve host: github.com
Warning: Problem : timeout. Will retry in 1 seconds. 9 retries left.
curl: (6) Could not resolve host: github.com
Warning: Problem : timeout. Will retry in 2 seconds. 8 retries left.
curl: (6) Could not resolve host: github.com
Warning: Problem : timeout. Will retry in 4 seconds. 7 retries left.
curl: (6) Could not resolve host: github.com
Warning: Problem : timeout. Will retry in 8 seconds. 6 retries left.
curl: (6) Could not resolve host: github.com
Warning: Problem : timeout. Will retry in 16 seconds. 5 retries left.
curl: (6) Could not resolve host: github.com
Warning: Problem : timeout. Will retry in 32 seconds. 4 retries left.
cc-=#=#   #   #                                        curl: (6) Could not resolve host: github.com
Warning: Problem : timeout. Will retry in 64 seconds. 3 retries left.
curl: (6) Could not resolve host: github.com
Warning: Problem : timeout. Will retry in 128 seconds. 2 retries left.
cccc-=O=#  #   #    #                                   curl: (6) Could not resolve host: github.com
Warning: Problem : timeout. Will retry in 256 seconds. 1 retries left.
```

If you experience a delay or the command seems to hang during execution, and the last line of output is as follows:

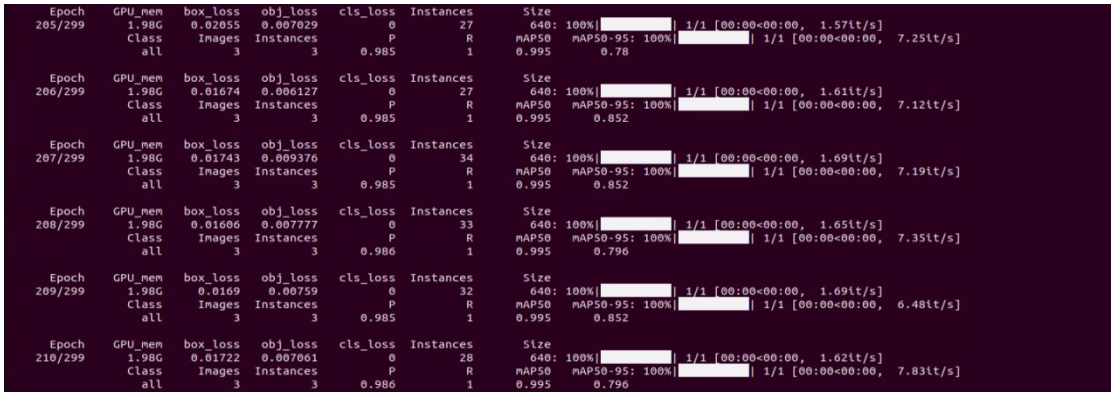
```
requirements: Ultralytics requirement ['opencv-python>=4.1.1'] not found, attempting AutoUpdate...
```

This indicates that the program has detected an incompatibility between the OpenCV version installed on your Raspberry Pi and the version required by the system, which has caused the program to pause and report the issue in the terminal. However, this issue does not affect the training of the model. You can choose either of the following solutions:

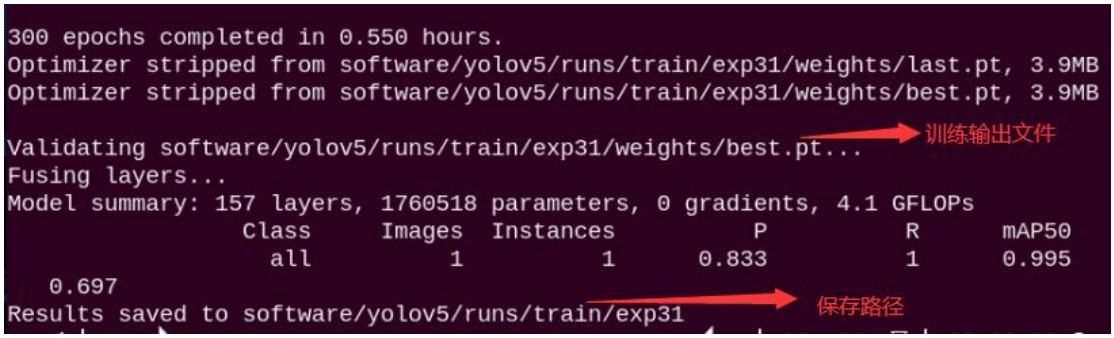
1. Simply press “**Ctrl+C”** to ignore the warning. The program will then continue executing.
2. Comment out the corresponding code in the script to prevent this message from blocking execution in future runs (file path: software/yolov5/train.py).

```
561 def main(opt, callbacks=Callbacks()):
562     # Checks
563     if RANK in {-1, 0}:
564         print_args(vars(opt))
565         check_git_status()
566         #check_requirements(ROOT / "requirements.txt")
```

If the output shown below appears, it indicates that the training is in progress.




After the model training is complete, the terminal will display the path where the generated files are saved. The results from this training session have been saved in the runs/train/exp directory under the yolov5 folder




**Note: The path to the generated files is not fixed. You can find the corresponding folder within the runs/train directory.**

## 4. Traffic Sign Model Training

**Please use the actual product names and reference paths as they appear in the document.**

For large datasets, it is not recommended to use the controller on the robot for training due to its slower training speed caused by I/O port speed and memory limitations. It is advisable to use a computer equipped with a dedicated graphics card for faster training. The training process remains the same, and you only need to configure the relevant program running environment.

If the performance of traffic sign recognition in the driverless game is poor and you need

to train your own model, you can refer to this section for training the traffic sign model. The screenshots in the following instructions may display different robot host names (though the environment configurations of different robots are roughly the same). Please input them according to the document content; however, it will not affect the execution.

## 4.1 Getting Ready

1) Prepare a laptop, or a PC with wireless network card and mouse.

2) Access the robot system desktop using VNC.

## 4.2 Training Instructions

### 4.2.1 Image Collecting

1) Start the robot, and access the robot system desktop using VNC.

2) Click-on  to open the command-line terminal.

3) Execute the command to terminate the app service.

**~/.stop_ros.sh**

```
~/.stop_ros.sh
```

4) Run the command to initiate the camera service.

**ros2 launch peripherals depth_camera.launch.py**

```
ros2 launch peripherals depth_camera.launch.py
```

5) Open a new terminal window. Navigate to the directory where the collection tool is stored, then enter the command to launch the image collection tool.

**cd software/collect_picture && python3 main.py**

```
cd software/collect_picture && python3 main.py
```


“save number” represents the picture ID, indicating which picture has been saved. “existing” denotes the total number of saved images.

6) Change the storage path to “/home/ubuntu/my_data”.


7) Position the target recognition content within the camera's field of view, then click the “Save (space)” button or press the space bar to capture and save the current camera image. Upon saving, both the “save number” and “existing” counts will increment by 1. These two parameters allow for monitoring of the saved picture names displayed on the current camera screen and the total number of pictures stored in the folder.


Upon selecting the "Save (space)" option, a JPEGImages folder will automatically be created within the directory "/home/ubuntu/my_data" to store the images.


---

Notice:

1. For enhanced model reliability, capture target recognition content from various distances, rotation angles, and tilt angles.

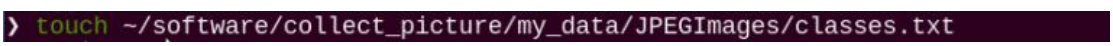
2. To guarantee recognition stability, it's advisable to increase the number of training images. It is recommended that each image category comprises at least 200 images for effective model training.

---

8) After image collecting, click-on “Exit” to close this software.


9) Create a new file by running the command:

```
touch ~/software/collect_picture/my_data/JPEGImages/classes.txt
```



### 4.2.2 Image Labeling

Note: the input command should be case sensitive, and keywords can be complemented using Tab key.

1) Start the robot, and access the robot system desktop using VNC.

2) Click-on  to open the command-line terminal.

3) Execute the command to terminate the app service.

**~/.stop_ros.sh**

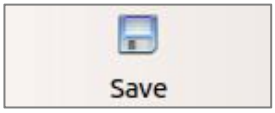
```
~/.stop_ros.sh
```

4) Create a new terminal to execute the command.

**python3 software/labelImg/labelImg.py**

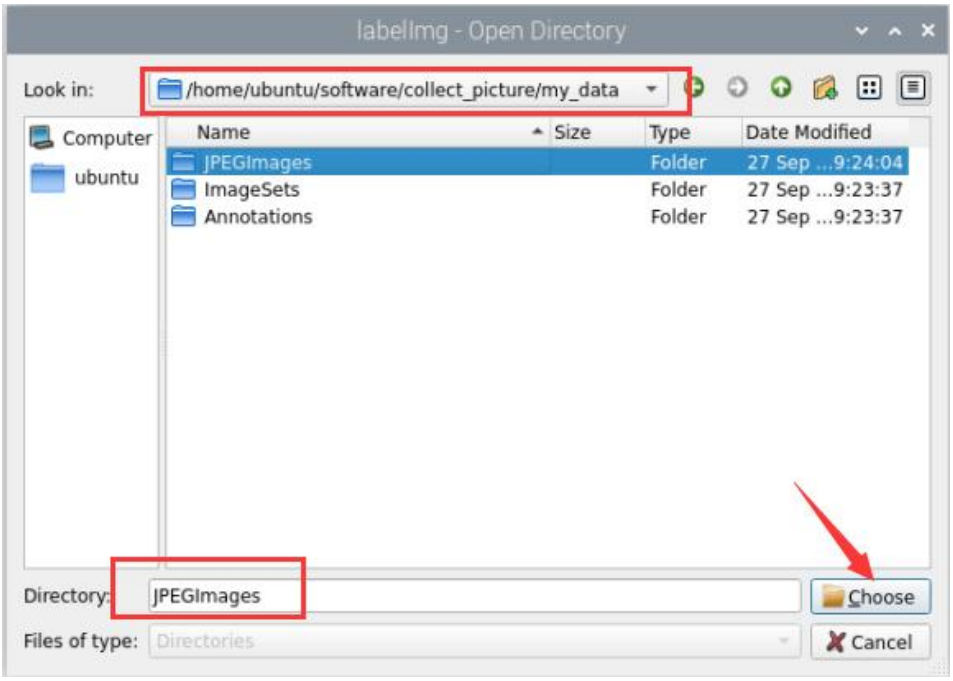
```
python3 software/labelImg/labelImg.py
```

5) Upon launching the image annotation tool, the table below outlines the key functions

| Icon | Shortcut Key | Function |
|---|---|---|
|   | Ctrl+U | Select the directory where the picture is saved. |
|   | Ctrl+R | Select the directory where the calibration data is saved. |
|   | W | Create annotation box |
|   | Ctrl+S | Save annotation |
|   | A | Switch to the previous image |
|   | D | Switch to the next image |

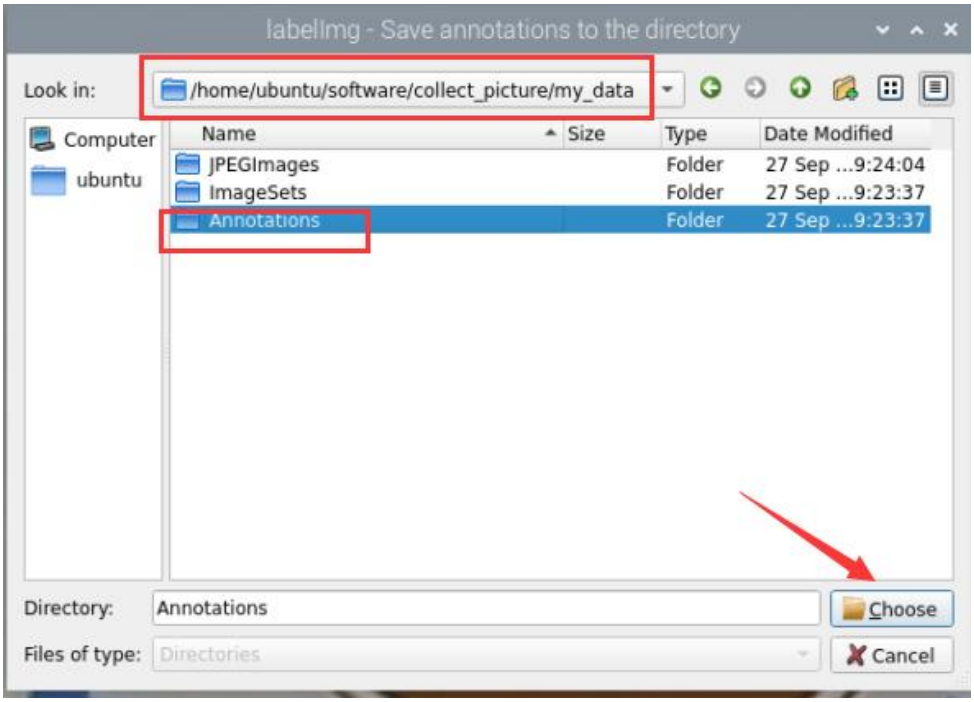
6) Click   to open the directory for image acquisition. In

this section, the selected path is shown in the figure below:




7) Click 

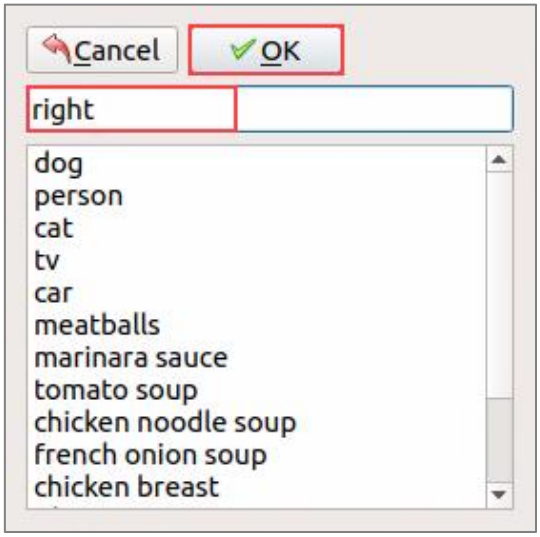
in the interface, and select the calibration data directory by choosing the corresponding storage folder. The path should be the “Annotations” directory located under the same root path as the image collection folder.



8) Press the "**W**" key to initiate label box creation.

Position the mouse cursor appropriately, then press and hold the left mouse button while dragging to encompass the entire target recognition content within the label box. Release the left mouse button to finalize the selection of the target recognition content.

9) In the pop-up window, assign a name to the category of the target recognition content, such as "right". Once the naming is complete, either click the "OK" button or press the "Enter" key to save this category.




10) Use short-cut '**Ctrl+S**' to save the labeled data of the current pictures.

11) Label the remaining pictures in the same manner as step 9).

12) Click  in the system status bar to open the file manager. Navigate to the directory "**/home/ubuntu/my_data/Annotations/**" to view the picture annotation file.

### 4.2.3 Generate Related Files

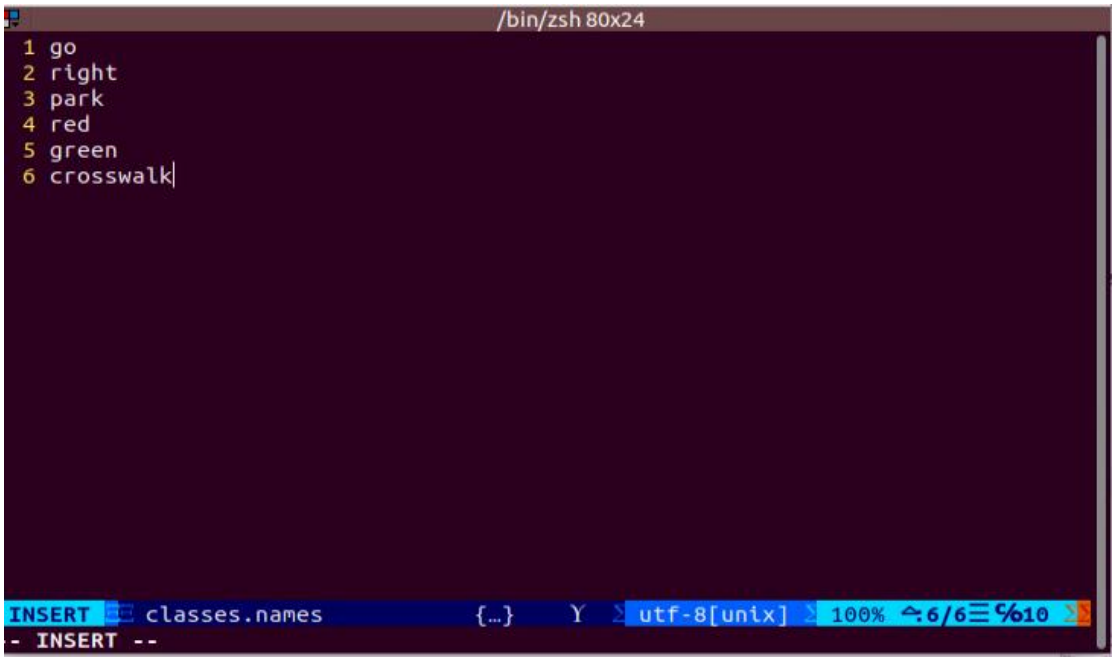
1) Click-on  to open the command-line terminal.

2) Execute the command to view the file.

```
vim software/collect_picture/my_data/classes.names
```

```
❯ gedit software/collect_picture/my_data/classes.names
```

3) Press the "i" key to enter the editing mode and input the class name of the target recognition content. If multiple class names are required, enter them one per line.




Note: The class name entered here must match the naming convention used in the image annotation software "labelImg" when applicable.

4) Having finish input, press '**Esc**' key, and input '**:wq**' to save the change and close the file.


5) Execute the command to convert the data format.

**python3 software/collect_picture/xml2yolo.py --data software/collect_picture/my_data --yaml software/collect_picture/my_data/data.yaml**

```
❯ python3 software/collect_picture/xml2yolo.py --data software/collect_picture/my_data --yaml software/collect_picture/my_data/data.yaml
data: software/collect_picture/my_data
turn_around:5
{'train': 'software/collect_picture/my_data/ImageSets/train.txt', 'val': 'software/collect_picture/my_data/ImageSets/val.txt', 'nc': 1, 'names': ['turn_around']}
finish!
```

The xml2yolo.py script converts annotated files into XML format, organizes the dataset into training and validation sets.

If you encounter the following prompt, the conversion is successful.


The output paths may vary on different robots based on their actual storage locations. However, the generated data.yaml file corresponds to the calibrated dataset.

6) Next, return to the command-line terminal, enter the command to edit the YAML file, and press Enter. For the modification method, please refer to section 2.2 above.

```
gedit software/collect_picture/my_data/data.yaml
```


7) Next, return to the command-line terminal, enter the command to edit the val.txt file, and press Enter. The modification method is the same as above.

```
gedit software/collect_picture/my_data/ImageSets/val.txt
```

8) Similarly, enter the command to edit the train.txt file, and press Enter.

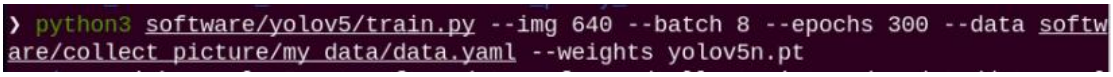
```
gedit software/collect_picture/my_data/ImageSets/train.txt
```


### 4.2.4 Model Training

1) Click-on  to open the command-line terminal.

2) Run the command to train the model.

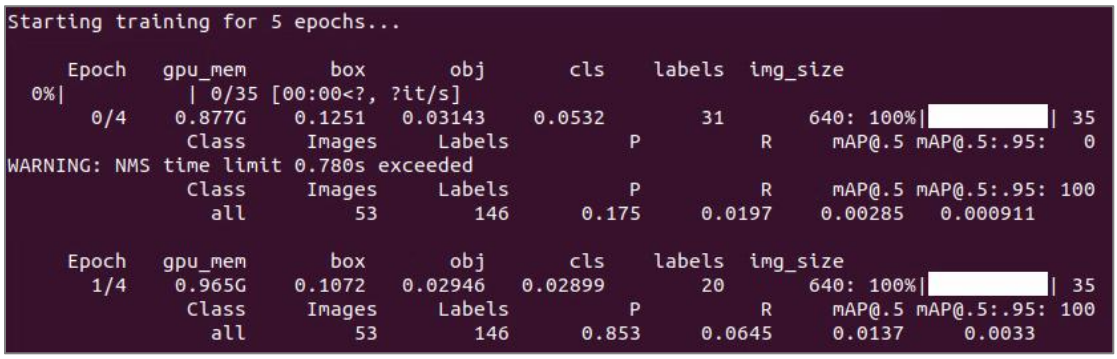
```
python3 software/yolov5/train.py --img 640 --batch 8 --epochs 300 --data software/collect_picture/my_data/data.yaml --weights yolov5n.pt
```



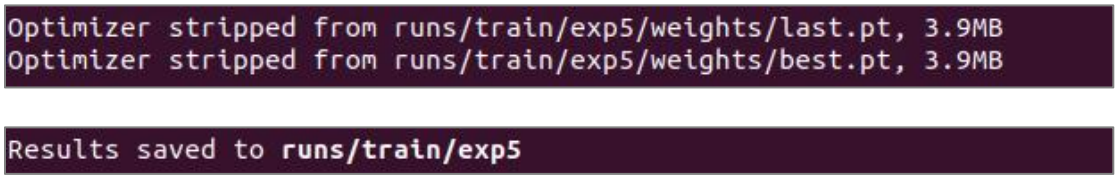

In the command, "--img" specifies the image size, "--batch" determines the number of images processed in each iteration, and "--epochs" denotes the number of training iterations, which impacts the quality of the final model. For quick testing, we've set the number of training iterations to 8, but for optimal results, this value should be adjusted based on your specific requirements and the capabilities of your computer system.

Furthermore, "--data" indicates the path to the calibrated dataset, while "--weights" refers to the path of the pretrained model. It's crucial to remember whether you're using "yolov5n.pt" or "yolov5s.pt" as your pretrained model.

You can customize these parameters according to your specific needs. If model reliability needs to be improved, consider increasing the number of training iterations, keeping in mind that this will also extend the training time. If the content displayed in the image below appears, it indicates that the training is ongoing.




After the model has been trained successfully, the terminal will print the path containing the file, and you need to record the path which will be required for “Generate TensorRt Model Engine”.







---

Notice: If multiple trainings are conducted, the naming of the "exp5" folder mentioned here will vary. For instance, it might be changed to "exp2", "exp3", and so on. The following steps will be associated with the folder naming used in this step.

---

## 4.3 Model Usage

1) Enter the command. Press “Enter” to disable the app service.

```
~/.stop_ros.sh
```


2) Execute the command to navigate to the file folder of training result.

```
cd software/yolov5/runs/train/exp5/weights
```

3) Copy the trained model to the ros2 function pack of the yolov5.

```
cp -r best.pt ~/ros2_ws/src/yolov5_ros2/config
```

```
cp -r best.pt ~/ros2_ws/src/yolov5_ros2/config
```

4) Run the command to open the depth camera node.

**ros2 launch peripherals depth_camera.launch.py**

```
ros2 launch peripherals depth_camera.launch.py
```

5) Open a new command line terminal. Execute the command to open the yolov5 node. Assign the current model as the copied model “best”.

**cd ros2_ws/src/yolov5_ros2/launch && gedit yolov5_ros2.launch.py**

```
cd ros2_ws/src/yolov5_ros2/launch && gedit yolov5_ros2.launch.py
```

As figure below, edit the parameter **model** in the opened page by changing its value to the model file name **best**:

Before modification:

Open *yolov5_ros2.launch.py
~/ros2_ws/src/yolov5_ros2/launch

```
# yolo_v5_ros2_launch.py

import os
import launch
from launch import LaunchDescription
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.actions import IncludeLaunchDescription, DeclareLaunchArgument, OpaqueFunction

def launch_setup(context):
# def generate_launch_description():
    package_share_directory = get_package_share_directory('yolov5_ros2')
#    peripherals_package_path = get_package_share_directory('peripherals')

    # depth_camera_launch = IncludeLaunchDescription(
        # PythonLaunchDescriptionSource(
            # os.path.join(peripherals_package_path, 'launch/depth_camera.launch.py')),
    # )
    yolov5_ros2_node = Node(
        package='yolov5_ros2',
        executable='yolo_detect',
        output='screen',
        parameters=[
            {"device": "cpu",
            #"model": "garbage_classification",
            "model": "traffic_signs_640s_7_0",
            'image_topic': '/ascamera/camera_publisher/rgb0/image',
            #"camera_info_topic": "/camera/camera_info",
            #"camera_info_file": f"{package_share_directory}/config/camera_info.yaml",
           # "show_result": True,
            "pub_result_img": True}

        ]
    )
```

After modification:

Open ▾ yolov5_ros2.launch.py ~/ros2_ws/src/yolov5_ros2/launch

```
# yolo_v5_ros2_launch.py

import os
import launch
from launch import LaunchDescription
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.actions import IncludeLaunchDescription, DeclareLaunchArgument, OpaqueFunction

def launch_setup(context):
# def generate_launch_description():
    package_share_directory = get_package_share_directory('yolov5_ros2')
#    peripherals_package_path = get_package_share_directory('peripherals')

    # depth_camera_launch = IncludeLaunchDescription(
        # PythonLaunchDescriptionSource(
            # os.path.join(peripherals_package_path, 'launch/depth_camera.launch.py')),
    # )
    yolov5_ros2_node = Node(
        package='yolov5_ros2',
        executable='yolo_detect',
        output='screen',
        parameters=[
            {"device": "cpu",
            #"model": "garbage_classification",
            #"model": "traffic_signs_640s_7_0",
            "model": "best",
            "image_topic": "/ascamera/camera_publisher/rgb0/image",
            #"camera_info_topic": "/camera/camera_info",
            #"camera_info_file": f"{package_share_directory}/config/camera_info.yaml",
           # "show_result": True,
            "pub_result_img": True}

        ]
    )

```

6) Execute the following command in the path specified in Step 5) to launch the yolov5 node.

Note: This command must be run only in the **/home/ubuntu/ros2_ws/src/yolov5_ros2/launch** directory.

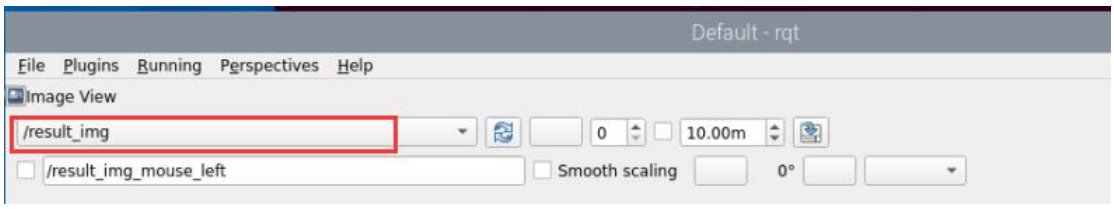
```
ros2 launch yolov5_ros2.launch.py
```

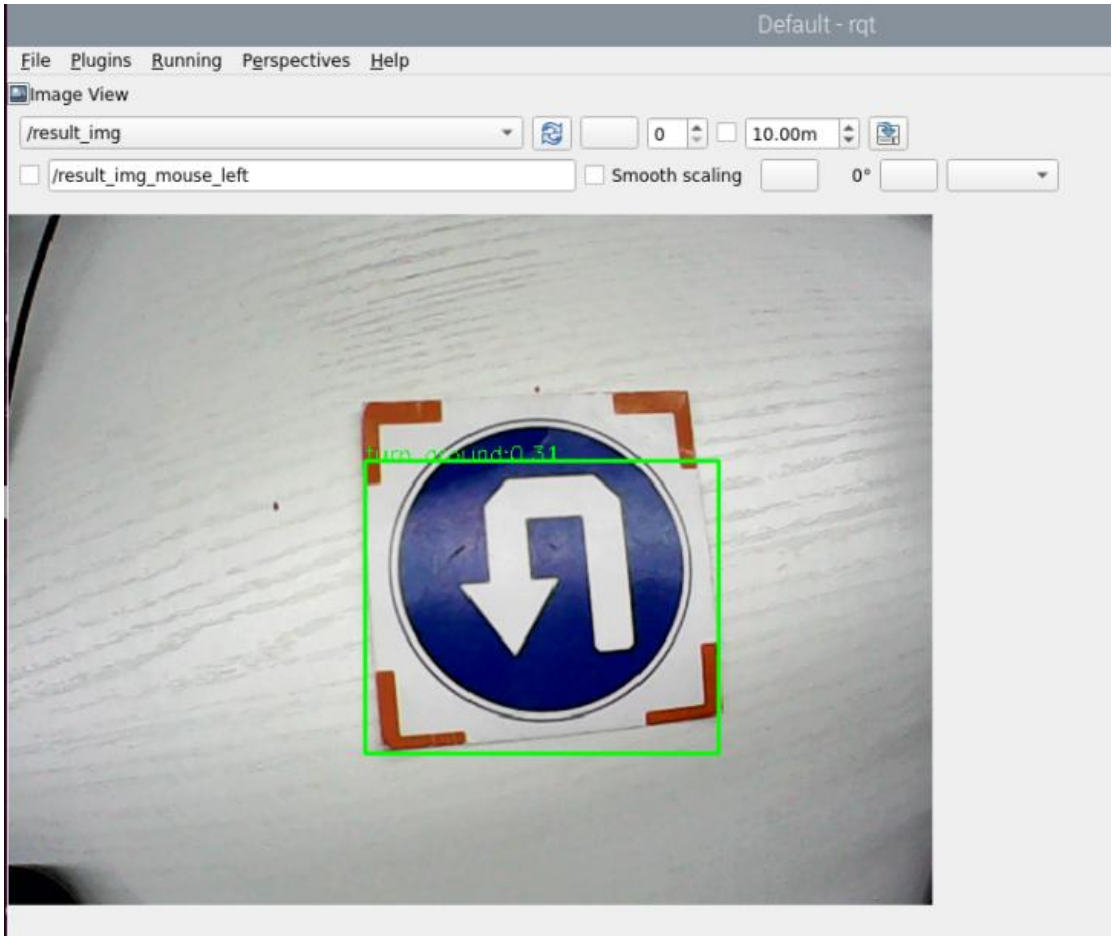
7) Open a new terminal and enter the command to launch the rqt interface:

```
rqt
```

```
❯ rqt
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/runtime-ubuntu'
```

8) In the rqt window, select the image source as “result_img”. Then, the window will display the real-time images captured by the camera and processed by the specified model. If you place the training materials in view, you will see them being detected and outlined with bounding boxes. Above each bounding box, the program’s recognition result and its confidence score will be displayed.




Default - rqt
File Plugins Running Perspectives Help
Image View
/result_img
0
10.00m
/result_img_mouse_left
Smooth scaling
0°